

Re-ranking with Late Interaction Models or Cross-Encoders

A Travel Recommender Search with Multi-Stage Vectors Embeddings

Start →



Journey is towards Semantic Search

Then Master Thesis

- ★ PROCESSING RDF Knowledge Graphs from Wikidata
- ★ PRE-PROCESSING Ant-optimization for pre-selection
- ★ HANDCRAFTED ONTOLOGY Hand-built ontology
- ★ SEARCH Simple Query Traversal
- ★ STRATEGY Bottom-up scoring

Now Qdrant Re-ranking Search

- ★ PROCESSING Unstructured text from WikiVoyage
- ★ PRE-PROCESSING Zero schema, embed and upsert (pydantic)
- ★ SEARCH Vector Emdebbings + Reranking
- ★ STRATEGY Top-down, bottom-up, and flat strategies

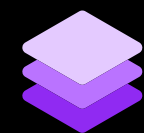
Multi-Granularity Recommendation

★ When choosing a travel destination, what drives the decision: an overall interest in the location itself, or the appeal of its specific points of interest?

My thesis answer: POI

Answer of this project: Both

Example search query: sea and ocean



City Layer: Naples, Rejeka, Kiel, Pula



POI Layer: "Diving around the coast of Rijeka"

Challenges & Solutions

★ NON FUNCTIONAL
A person wants to get both city and its sights

★ FUNCTIONAL
Simple Embeddings can not usually provide high accuracy

★ FUNCTIONAL
People usually use convoluted and ambiguous language

★ ML CLASSICS
Free data is scattered and is of poor quality

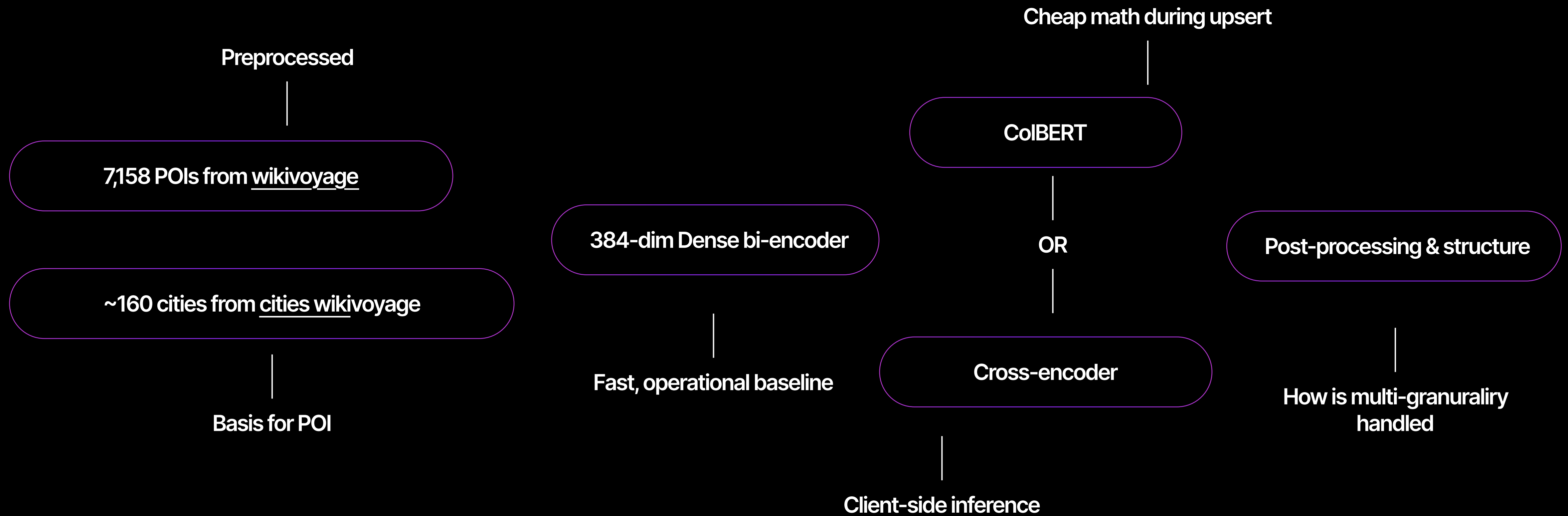
★ NON FUNCTIONAL
Search architecture that will account for both dimensions

★ FUNCTIONAL
Use Cross-encoding or Late-interaction model to increase accuracy

★ FUNCTIONAL
Vector embedding similarity search

★ ML CLASSICS
Restrict the area to available dataset - defensive but strong starting point

Simplicity to win them all



Search Strategies

FASTEST



Simple (dense only)

Flat search across all documents, no reranking

MOST HONEST



Flat cross-encoder

Dense retrieval + client-side reranking, POIs only

SURPRISING



Bottom-up

Dense retrieval + flat ColBERT rerank → group results by city post-hoc

SURPRISING



Top-down

Dense + ColBERT finds cities first → dense + ColBERT finds POIs within each winning city

Live Demo

```
+ Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ... .venv (Python 3.14.4)
```

Query 1: sea and oceans

Really ambiguous and abstract search query. Lets see how each strategy can handle it

```
sea_and_oceans: Any = search_queries.at[0, "query"]
sea_and_oceans
```

[44] ✓ 0.0s Python

... 'sea and oceans'

```
create_search_result(query=sea_and_oceans)
```

[45] ✓ 0.8s Python

... === SIMPLE_SEARCH (136.3ms) ===

```
[score=0.8773] Fish at the Baltic sea in Kaliningrad (do): All year round
[score=0.8462] Ordu in Samsun (other): – on the way to more popular cities and sites of eternally rainy and green Eastern Black Sea
[score=0.8402] L’aparte in Baku (eat): European and Azeri Cuisine
[score=0.8363] Asia in Zagreb (eat): Chinese restaurant
[score=0.8304] Museum of the World Ocean in Kaliningrad (see): Includes two museum ships and one submarine.
[score=0.8292] El Cormorán in Santander (eat): dining by the sea
[score=0.8257] Dover Strait in Burgas (drink): Bar & cigars
[score=0.8252] Ivanovskiy Bridge in Astrakhan (other): Over the First of May canal
[score=0.8234] Bol d'Or in Geneva (do): Yacht Race (biggest in Europe).
[score=0.8233] Respublika Luks in Baku (eat): European and Azeri Cuisine
[score=0.8230] Murcia: You could be looking for:
[score=0.8228] Valladolid: You may be looking for:
```

Query results

Query	Spec.	Facets	Intent	Ranking	Key Insight
sea and ocean	abstract	single	discover	top-down > bottom-up > encoder > simple	city descriptions capture geography better than POIs
romantic dinner + architecture	moderate	multi	discover	top-down > bottom-up > encoder > simple	facets split naturally across hierarchy levels
medieval fortifications	specific	single	discover	bottom-up ≈ encoder ≈ simple > top-down	all methods work when vocabulary directly matches
ancient ruins & archaeology	specific	single	discover	bottom-up > encoder > simple > top-down	rich POI descriptions don't need city-level help
hostel wifi train in Kiel	specific	multi	explore	top-down ≈ encoder > simple > bottom-up	named city acts as hard constraint
relaxing escape + nature + food	abstract	multi	discover	all weak (bottom-up ≈ simple > top-down > encoder)	mood-based queries hit data vocabulary mismatch
extreme sports and adrenaline	specific	single	discover	simple > bottom-up > top-down > encoder	rare topic scattered across cities, grouping hurts
star gazing	abstract	single	discover	all fail (simple > others)	polysemy — "star" dominantly means hotel rating
peaceful place to party	abstract	multi	discover	bottom-up > simple > encoder > top-down	ColBERT resolves contradiction via independent token matching

Patterns of each model

TOP DOWN

- ✦ Wins when geography matters. In general is good with abstract queries. Downsides are that many times abstracts of cities are too generic for concrete or contradictory queries

BOTTOM UP

- ✦ Bottom up wins when POI already has the answer. ColBERT's token-level matching resolves even contradictory queries ("peaceful place to party") however its flatness does harm it when context of the whole region is important

SIMPLE QUERY

- ✦ Flat/simple search wins when relevant results are sparsely distributed. Grouping in those cases harms the results (extreme sports). Otherwise this model has more false positives than others

CROSS ENCODER

- ✦ Cross encoder of this dimensionality needs more precise corpus. Most honest model but also most rigid from the results

However, all methods fail when corpus
is flooded with false positives – no
deep semantic understanding

Star gazing exposed this blind spot - all models thought its a hotel name, syntactically correct, however
semantically far from ideal

Sentiment based queries can be fragile like in case of “relaxing weekend”. No data in corpus for that

Late Interaction Model vs Cross Encoder

	Simple Model	Late Interaction	Cross-encoder
Latency	Very Low (~200ms)	Low (~400ms)	High (~1000ms)
Storage cost	1x - 1 vector	10-25x - 2 vectors	1x - 1 vector
Precision	Low	High	Very High
Conservative	Low	Low	High
Architectural flexibility	Low	High	Low
Search cost	Low	Low	High

Late Interaction Model is preferable
where data is less reliable, latency is
an issue and good enough precision is
needed

Thank you for listening

Done by Cristea Andrei

GitHub: <https://github.com/havenusername/qdrant-reranker>

Built with: Qdrant (open-source) + FastEmbed + Pydantic on Python

Deployment: Env variable change to point Qdrant Cloud endpoint